

Proyecto 3: Chatbot Inteligente con Programación Lógica

El propósito de este proyecto es aplicar los principios del paradigma lógico mediante el desarrollo de un chatbot inteligente utilizando el lenguaje Prolog. El proyecto permitirá desarrollar habilidades relacionadas con representación del conocimiento, inferencias lógicas, reglas declarativas, manejo dinámico de hechos, procesamiento básico de lenguaje natural y generación de nuevo conocimiento.

El sistema deberá ser capaz de mantener conversaciones simples por consola, responder preguntas utilizando una base de conocimiento inicial y aprender nueva información durante la ejecución.

Descripción del programa

2.1 Base de Conocimiento Inicial

El programa debe:

- Contener una base de conocimiento inicial compuesta por reglas y hechos lógicos.
- La base de conocimiento debe permitir responder preguntas sobre diferentes temas.
- Cada integrante del grupo deberá desarrollar al menos:
 - 50 reglas/hechos/conocimientos iniciales.
- El sistema deberá soportar:
 - conceptos,
 - relaciones,
 - definiciones,
 - sinónimos,
 - asociaciones lógicas.
- Ejemplos:

```
concepto(prolog, 'Lenguaje de programación lógica').  
es_un(perro, animal).  
sinonimo(programacion_logica, prolog).
```

2.2 Conversación con el Usuario

El chatbot deberá funcionar completamente mediante consola.

- El usuario podrá:
 - realizar preguntas,
 - solicitar definiciones,
 - conversar con el chatbot,
 - enseñar nueva información,

- enseñar sinónimos,
 - consultar conocimiento existente.
- Ejemplos:
 - ¿Qué es Prolog?
 - Explique recursividad
 - Aprender que un gato es un animal
 - Aprender que IA significa inteligencia artificial
- El sistema debe responder utilizando inferencias y coincidencia de patrones.

2.3 Aprendizaje Dinámico

- Si el chatbot no conoce una respuesta, deberá:
 - indicar que no posee conocimiento suficiente,
 - solicitar al usuario una posible respuesta o definición,
 - almacenar el nuevo conocimiento dinámicamente.
- Se sugiere utilizar la meta:
 - `assertz()`

para agregar nuevos hechos y reglas durante la ejecución.

- Ejemplo:
 - `assertz(concepto(haskell, 'Lenguaje funcional'))`.
- El chatbot debe ser capaz de reutilizar posteriormente el conocimiento aprendido.

2.4 Aprendizaje de Sinónimos

- El sistema deberá permitir que el usuario enseñe sinónimos o relaciones equivalentes.
- Ejemplos:
 - Aprender que IA es sinónimo de inteligencia artificial
 - Aprender que PL significa programación lógica
- Ejemplo de representación:
 - `sinonimo(ia, inteligencia_artificial)`.
- El chatbot deberá utilizar dichos sinónimos para mejorar la interpretación de preguntas.

2.5 Variaciones de Preguntas

- El sistema no debe depender únicamente de preguntas exactas.
- Debe ser capaz de interpretar variaciones similares mediante:
 - coincidencia de palabras,
 - detección de palabras clave,
 - manejo básico de sinónimos,
 - patrones lógicos.
- Ejemplos válidos:

- ¿Qué es Prolog?
 - Explique Prolog
 - Defina Prolog
 - ¿Para qué sirve Prolog?
- Todas deberían producir respuestas relacionadas con el mismo concepto.

2.6 Inferencias Lógicas

- El chatbot deberá utilizar reglas declarativas para generar conclusiones simples.
- Ejemplo:
 - `es_un(perro, animal).`
 - `tiene(animal, vida).`
- Entonces el sistema podría inferir:
 - El perro tiene vida.
- El objetivo es aplicar razonamiento lógico y relaciones entre hechos.

2.7 Flujo Conversacional

- Toda la lógica del chatbot debe desarrollarse utilizando:
 - reglas declarativas,
 - recursividad,
 - backtracking,
 - predicados dinámicos.
- El uso de ciclos imperativos está prohibido.
- El sistema debe:
 - mantener la conversación activa,
 - permitir múltiples consultas,
 - permitir aprendizaje dinámico,
 - finalizar únicamente cuando el usuario indique salir.

Funciones por investigar

Para implementar el programa, se recomienda investigar:

- Manejo de listas y predicados recursivos en Prolog.
- Uso de predicados dinámicos:
 - `assert/1`
 - `assertz/1`
 - `retract/1`
- Entrada y salida básica:
 - `read/1`
 - `write/1`
 - `nl/0`
- Procesamiento básico de strings y listas.

- Comparación de patrones.
- Manejo de sinónimos y coincidencias.
- Representación del conocimiento.
- Inferencias lógicas simples.

Aspectos técnicos

- El proyecto debe ser desarrollado en SWI-Prolog u otro intérprete compatible.
- Debe ejecutarse desde línea de comandos, sin necesidad de interfaz gráfica.
- El código debe ser modular, comentado y estructurado.
- Puede usar predicados auxiliares siempre que estén bien documentados.

Aspectos Administrativos

- La tarea vale un **15%** de la nota del curso
- La tarea se hará en grupos de **4 personas máximo**.
- Fecha de entrega: **Jueves 4 de Junio de 2026**. No se aceptan tareas entregadas después de esa fecha.
- Los grupos deberán subir el código y la documentación de sus respectivas tareas a un repositorio en Github, de manera que el profesor pueda ver las contribuciones que las diferentes personas hacen al proyecto. La idea es que apenas empiecen a desarrollar la tarea, suban las contribuciones al repositorio, y no esperar a tener todo el código listo para subirlo.
- En el TEC Digital, deberán subir un documento con las siguientes secciones:
 - Portada
 - Índice
 - Enlace de GitHub
 - Pasos de instalación del programa
 - Manual de usuario
 - Arquitectura lógica utilizada
 - Explicación del funcionamiento
- Todos los miembros del grupo deberán participar de la revisión, ya que de lo contrario no se les asignará el puntaje correspondiente. La nota de la revisión es individual, el resto de la nota es grupal.
- El código entregado debe ser 100% original. En caso de probarse algún tipo de fraude en la elaboración de la tarea, se aplicarán todas las medidas correspondientes, según el reglamento del TEC, incluyendo una carta al expediente.

Evaluación

Aspecto por evaluar	Porcentaje
Documentación	10%
Base de conocimiento inicial	20%
Manejo conversacional	15%
Aprendizaje dinámico	25%
Manejo de sinónimos y variaciones	15%
Inferencias lógicas	15%